

Atividade Prática Assíncrona

Geomerez Raduan de Oliveira Bandeira

Repositório: https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona. O detalhe das etapas está em [ia-dev-lab/docs/relatorio-harness-etapas.md](#). O histórico desta prática está no Pull Request https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona/pull/3.

1. Autonomia e guardrail. Executei a mesma tarefa (prazo opcional e toaster para o dia seguinte) nos modos Auto e Plan, com Composer. O Auto (feature/prazo-auto) concluiu em cerca de trinta minutos, com retrabalho nos testes e risco médio. O Plan (feature/prazo-plan) exigiu aceite do plano escrito e levou cerca de quarenta minutos; o risco percebido foi baixo. Prefiro o Auto só no tempo. Prefiro o Plan no controle e segui com essa branch. O hook em `.cursor/hooks.json` bloqueia `migrate:fresh`, `db:wipe` e a remoção do SQLite do laboratório. Na tentativa com o agente, o Cursor recusou a ação.

2. TDD e enforcement. No filtro `all|pending|done`, o ciclo foi teste, falha e código mínimo: Red 49ca1d1, Green 48a55fe, regra no Service. Sem TDD, a edição de título respeitou as camadas, mas a suíte antiga ficou verde sem cobrir o rename. Na prioridade usei o Superpowers de ponta a ponta: desenho aprovado, depois o teste que falhou e o código até voltar a verde (0b75d75). O plugin reforçou o `AGENTS.md`; não o substituiu.

3. Checkpoint humano. O transcript está em [ia-dev-lab/docs/sessao-log.md](#). O ponto de parada é antes de criar ou aplicar migration. Isso não se confunde com o hook: o hook recusa destruir o banco; o checkpoint decide se o schema muda. Na simulação do Plan do `due_date`, a decisão foi aprovar. Assumi o papel de responsável pelo schema e pelos dados do SQLite no Docker, não só de revisor de código.

4. Decisão, ADR e diagrama. O `ToDo` está no tamanho certo: extrair API de tarefas ou de login viraria acoplamento de rede sem segundo consumidor. Fechei o contrato da aplicação com `app/Domain`. O ADR 0005 registra isso no formato MADR; os ADRs 0001 a 0004 permaneceram intactos. No C4 de contêiner, a versão B comunica melhor, porque desenha o que se implanta (tela, API, SQLite, Google e GitHub), não a camada POSA.

5. Custo e tokens. Estimei pelos transcripts (caracteres do log divididos por 4). A prioridade consumiu cerca de 22 mil tokens; o filtro, o mesmo patamar; o prazo, cerca de 58 mil, porque foram Auto, Plan e hook. Reescrever o relatório custou mais que o filtro. A estratégia aplicável é *model routing*: Composer na feature e modelo menor nos ajustes de LaTeX.

6. Uma dificuldade real. A dificuldade foi conceitual, no C4 de contêiner. O prompt curto promoveu Service e repositório a processo, inventou MySQL e omitiu o GitHub. Camada POSA não é unidade de implantação neste laboratório: Service e repositório vivem dentro da API. Quem lê só a versão A pode achar que existe um microserviço de regra e outro de dados. Só o segundo prompt, dizendo o que *não* é contêiner, descreveu a arquitetura do `ToDo`.